

Analisa Coding Aplikasi Rental Mobil

1. Ringkasan Umum

Apakah implementasi sudah sesuai dengan temuan analisa?

Secara garis besar: **Ya, sebagian besar sesuai**, namun ada beberapa gap kritis terutama pada implementasi RAG yang seharusnya menjadi inti sistem.

Teknologi yang Ditemukan:

- **Backend:** Laravel (PHP) — terlihat dari struktur `app/Http/Controllers`, `routes/web.php`, `models`, `migrations`, `config`
- **Database:** MySQL (`if0_41735785_rental_mobil` di `sql212.infinityfree.com`), SQLite lokal
- **AI/RAG Engine:** Python Flask eksternal di `https://iZikra-iZikraSpaces.hf.space` (via `env RAG_ENGINE_URL`)
- **Session:** File-based (tidak Redis di production)
- **Frontend:** Blade templates + Vanilla JS (tidak ada React/Vue)

Gap Terbesar:

RAG retrieval yang sesungguhnya (vector similarity search via ChromaDB) tidak terlihat di kode Laravel. Yang terjadi adalah **context injection dari MySQL ke prompt LLM** — ini lebih tepat disebut *prompt engineering dengan context* bukan RAG sungguhan.

2. Tabel Perbandingan

Poin Analisa	Status	Bukti di Kode	Keterangan
Multi-tenant dengan rental_id	☒ Sudah diselesaikan	Mobils, Rentals, Transaksis memiliki rental_id; filter where('rental_id', \$rentalId) diterapkan di ChatbotController, MitraController, MitraMiddleware	Isolation tenant dilakukan dengan konsisten
URL parameter ?i= untuk tenant	☒ Sudah diselesaikan	routes/web.php: session(['tenant_id' => \$tenantId]) dari \$request->query('tenant_id')	Param bisa tenant_id atau mitra
Session tracking tenant	☒ Sudah diselesaikan	resolveRentalId() di ChatbotController membaca session('tenant_id')	双重 proteksi: session + query param

Mitra lihat data miliknya sendiri	☒ Sudah diselesaikan	IsMitra middleware + authorization check di konfirmasiPesanan(), tolakPesanan(), selesaikanPesanan()	Otorisasi berlapis (owner pusat + admin cabang)
Chatbot endpoint aktif	☒ Sudah diselesaikan	Route::post('/bot/send-message', [ChatbotController::class, 'sendMessage'])->name('chatbot.send')	Terdapat di routes/web.php
Fallback jika AI gagal	☒ Sudah diselesaikan	buildOfflineReply() fallback di blok catch ChatbotController	Memberikan respons berbasis query MySQL langsung
Chat logs di database	☒ Sudah diselesaikan	ChatLog model + migration 2026_01_31_191346_create_chat_logs_table.php; ChatLog::create() dipanggil setiap chat	Menyimpan user_id, session_id, message, response, rental_id, model_used
ChromaDB untuk retrieval	 Sebagian diselesaikan	ChromaDB ada di luar Laravel (di HuggingFace Spaces service). Kode Laravel memanggil endpoint Flask /chat dan /search	ChromaDB TIDAK terlihat langsung di kode Laravel — ini service terpisah
Vector similarity search	☒ Belum diselesaikan	Tidak ada kode chroma, similarity_search, embedding di Laravel	Service Flask yang harusnya melakukan ini tidak bisa diverifikasi dari sisi Laravel
Metadata filtering (rental_id, kota)	 Hanya sebagian	Di Laravel: rental_id difilter, kota difilter via whereHas('branch')	Tapi filtering ini terjadi di MySQL query, bukan di ChromaDB query
Hybrid retrieval (vector + keyword)	☒ Belum diselesaikan	Tidak ada implementasi hybrid di kode	Kemungkinan dilakukan di service Flask, tapi tidak terlihat
Knowledge base untuk SOP/denda/harga	☒ Belum diselesaikan	Tidak ada evidence knowledge base dokumen (PDF/TXT) di storage/app/public	Tidak ada folder documents atau knowledge yang ditemukan
Booking online	☒ Sudah diselesaikan	TransaksiController::store(), route /order/simpan	Terintegrasi dengan Midtrans payment gateway
Guest booking dengan token/UUID	☒ Sudah diselesaikan	GuestBookingController dengan booking_token UUID + token_expires_at	Token expiry 30 menit, divalidasi dengan benar

Konfirmasi booking oleh mitra	☒ Sudah diselesaikan	MitraController::konfirmasiPesanan() dengan otorisasi multi-tenant	Notifikasi WA via Fonnte API
Status ketersediaan real-time	☒ Sudah diselesaikan	getBookedCarIds() menghitung semua mobil dengan status 'disewa', 'pending', dll	Mobil dengan token_expires_at > now() (draft expired) tidak dihitung
Navigasi lintas kota	☒ Sudah diselesaikan	Filter \$query->whereHas('branch', fn(\$q) => \$q->where('kota', \$request->kota)) di home route	Juga dideteksi otomatis dari pesan chat
Dashboard mitra	☒ Sudah diselesaikan	MitraController::dashboard() + view mitra.dashboard	Menampilkan total armada, pesanan aktif, pendapatan
Smart search	☒ Sudah diselesaikan	ChatbotController::smartSearch() endpoint /bot/smart-search	Filter: kota, jumlah_kursi, bahan_bakar, tahun
Comparison harga	⚠ Hanya sebagian	Smart search menampilkan daftar harga terurut (sortBy harga) tapi tidak ada fitur perbandingan 2 mobil side-by-side	Fungsi sortBy di buildOfflineReply() sudah ada
Filter kapasitas kursi	☒ Sudah diselesaikan	detectSeatsFromMessage() + query filter di buildOfflineReply() dan smartSearch()	Deteksi dari pesan: "mobil 7 kursi"
Filter BBM	☒ Sudah diselesaikan	detectFuelFromMessage() + filter bensin/solar/listrik	Konversi bensin → pertalite/pertamax/bensin
Filter tahun kendaraan	☒ Sudah diselesaikan	detectYearFromMessage() + filter tahun >=	Deteksi regex \b(20\d{2})\b

3. Poin yang Sudah Diselesaikan

Multi-Tenant

- rental_id sudah ada di mobils, rentals, transaksi, users (confirmed via model definitions dan migrations)
- Isolasi data dilakukan dengan filter where('rental_id', \$rentalId) yang konsisten di seluruh controller
- Otorisasi ganda: owner rental (tanpa branch_id) dan admin cabang (dengan branch_id) diperlakukan berbeda
- Session-based tenant tracking dengan fallback ke URL parameter

Sistem Chatbot

- Endpoint `/bot/send-message` sudah ter-register di `routes/web.php`
- Context building dari MySQL (stok mobil real-time) sudah baik: membangun `contextData` string dengan detail mobil (ID, unit, mitra, cabang, alamat, harga, transmisi, BBM, kapasitas)
- Fallback `buildOfflineReply()` berfungsi jika Flask engine gagal — menggunakan query langsung ke MySQL
- Chat log disimpan permanen ke tabel `chat_logs` via `ChatLog::create()`
- History diambil dari DB dengan `take(6)` dan reverse untuk context percakapan

Booking & Rental

- Guest booking flow lengkap: UUID token → form submission → transaksi created → WA notification
- Konfirmasi/tolak/selesaikan pesanan dengan otorisasi tenant yang ketat
- Status mobil di-update secara real-time saat konfirmasi (`status = 'disewa'`) dan saat selesai (`status = 'tersedia'`)
- Payment gateway Midtrans terintegrasi (SNAP token, webhook)
- WA notification ke customer dan mitra via Fonnte API

Fitur Pencarian

- Smart search dengan filter multi-dimensi (kota, kursi, BBM, tahun, transmisi)
- Detection otomatis parameter dari pesan natural: `detectCityFromMessage()`, `detectTransmissionFromMessage()`, `detectSeatsFromMessage()`, `detectFuelFromMessage()`, `detectYearFromMessage()`
- Sort hasil berdasarkan harga (`sortBy(fn($m) => (float) ($m->harga_sewa ?? 0))`)
- Booking link langsung di-generate per mobil hasil pencarian

Dashboard & Navigation

- Home page mendukung filter kota dan mitra via query string
- Dashboard admin pusat: statistik agregat seluruh sistem
- Dashboard mitra: statistik per rental dengan proteksi data tenant

4. Poin yang Belum / Belum Sepenuhnya Diselesaikan

4.1 RAG Retrieval Sejati dengan ChromaDB (KRUSIAL)

Masalah: Kode Laravel hanya membangun `contextData` string dari MySQL dan mengirimnya ke service Flask (`RAG_ENGINE_URL`). Tidak ada kode yang secara eksplisit:

- Menghubungi ChromaDB instance
- Memanggil `collection.query()` dengan `query_embeddings`
- Melakukan vector similarity search
- Mengambil dokumen dari ChromaDB berdasarkan skor kemiripan

Bukti tidak langsung:

- File `.env`: `RAG_ENGINE_URL=https://iZikra-iZikraSpaces.hf.space` — ini adalah service HF Spaces, bukan ChromaDB langsung
- Tidak ada konfigurasi `CHROMA_*` env variables
- Tidak ada library ChromaPHP atau chromadb client di `vendor/` (tidak bisa diverifikasi dari luar, tapi tidak ada require di `composer.json`)

Implikasinya: Apa yang disebut "RAG" saat ini adalah **in-context learning** — menyisipkan data MySQL sebagai konteks prompt ke LLM. Ini BUKAN RAG sejati karena tidak ada retrieval berbasis vector similarity.

File yang seharusnya ada (jika RAG benar):

- Konfigurasi ChromaDB client di Laravel
- Fungsi `chroma_client = chromadb.HttpClient(...)`
- `collection.query(query_embeddings=[...], n_results=5, where={"rental_id": ...})`
- Penggabungan hasil ChromaDB + MySQL (hybrid)

4.2 Metadata Filtering di ChromaDB

Even if ChromaDB were queried, filtering by `rental_id` and `kota` at the vector search level is not implemented. The `where` clause in ChromaDB query would be needed for true multi-tenant RAG.

4.3 Hybrid Retrieval (Vector + Keyword)

True hybrid retrieval would require:

1. Vector search: `collection.query(query_embeddings=[...])` → ChromaDB
2. Keyword search: BM25 / TF-IDF pada document store
3. RRF (Reciprocal Rank Fusion) untuk menggabungkan hasil

Tidak ada evidence ini di kode Laravel maupun konfigurasi.

4.4 Knowledge Base untuk SOP/Denda/Harga

Tidak ada folder `storage/app/public/documents` atau `storage/app/private/knowledge` yang terdeteksi. Tidak ada file SOP, daftar harga, syarat sewa, atau matriks denda yang termuat dalam sistem.

Implikasi: Chatbot yang menjawab pertanyaan tentang denda, SOP, atau harga kemungkinan besar **mengandalkan pengetahuan LLM saja** (parametric knowledge), bukan dari knowledge base retrieval.

5. Saran Perbaikan

Prioritas 1 — Perbaiki RAG Implementation (Jika Ini Skripsi)

File yang perlu diubah: Butuh akses ke kode Flask service di HuggingFace Spaces untuk diperbaiki. Jika itu di bawah kontrol, berikut yang harus ditambahkan:

1. **Di Flask service (`app.py` atau `rag_engine.py`):**

```
import chromadb
from chromadb.utils import embedding_functions

chroma_client = chromadb.HttpClient(
```

```

host=os.getenv('CHROMA_HOST', 'localhost'),
port=int(os.getenv('CHROMA_PORT', 8000))
)
collection = chroma_client.get_collection("rental_knowledge")

def retrieve_context(query: str, rental_id: str = None, top_k: int = 5):
    # 1. Vector search
    vector_results = collection.query(
        query_texts=[query],
        n_results=top_k,
        where={"rental_id": rental_id} if rental_id else None
    )
    # 2. Keyword search (BM25)
    keyword_results = bm25_search(query, top_k)
    # 3. Hybrid fusion (RRF)
    fused = reciprocal_rank_fusion([vector_results, keyword_results])
    return fused

```

2. Di Laravel (ChatbotController.php):

- Endpoint /chat harus mendapatkan hasil retrieval dari ChromaDB, bukan hanya context string MySQL
- Kirim query_embedding atau biarkan Flask yang generating embed di server

Catatan: Saat ini HuggingFace Spaces digunakan sebagai endpoint tunggal — jika Spaces tersebut melakukan query ke ChromaDB sendiri, maka verifikasi aksesnya butuh akses ke kode Spaces.

Prioritas 2 — Knowledge Base Ingestion Pipeline

Jika aplikasi benar-benar ingin memiliki RAG dengan knowledge base:

1. Buat folder storage/app/private/knowledge/ atau storage/app/public/documents/
2. Buat script ingestion:

```

// database/migrations/..._ingest_knowledge_base.php
// Loop semua file .txt/.pdf di folder knowledge
// Chunk dokumen → embed → store ke ChromaDB

```

3. Jenis dokumen: SOPRental.txt, DaftarHarga.txt, SyaratKetentuan.txt, Denda.txt
4. Metadata ChromaDB: {rental_id, doc_type, kota}

Prioritas 3 — Chat Log Enhancements

Chat log sudah ada, tapi bisa diperkuat:

- Tambahkan kolom `embedding_vector` di `chat_logs` jika ingin menyimpan kontekstual chat
- Index pada `chat_logs`: `(user_id, rental_id)`, `(session_id)`
- Tambah query builder `whereBetween('created_at', [...])` untuk filtering historis

Prioritas 4 — Guest Booking UI/UX

Guest booking flow sudah solid secara backend. Yang perlu diverifikasi:

- Apakah `resources/views/frontend/guest_booking.blade.php` ada dan sesuai
- Apakah upload foto identitas/SIM berfungsi (saat ini store ke `storage/app/public/identitas` dan `sim`)

6. Kesimpulan Akhir

Apakah aplikasi sudah layak dianggap implementasi skripsi yang sesuai?

Secara struktural: Ya — aplikasi memiliki arsitektur yang cukup lengkap untuk skripsi dengan topik Rental Mobil + AI Chatbot.

Secara teknis RAG: Belum sepenuhnya — istilah "RAG" yang dipakai dalam temuan analisa belum terealisasi secara sesungguhnya. Yang ada saat ini adalah:

- **In-context learning** (menyisipkan data MySQL sebagai prompt context)
- **Bukan retrieval-augmented generation** karena tidak ada vector similarity search yang confirmed

Bagian Paling Krusial untuk Segera Diperbaiki

1. **Verifikasi service Flask di HuggingFace Spaces** — Apakah benar-benar melakukan ChromaDB query? Jika tidak, ini harus diperbaiki di sisi Spaces. Kode Flask harus melakukan `collection.query()` dengan embeddings.

2. **Knowledge base documents** — Jika aplikasi mengklaim memiliki "RAG-based chatbot", maka knowledge base SOP/harga/denda harus ada sebagai dokumen yang di-ingest ke ChromaDB. Tanpa ini, chatbot tidak bisa menjawab pertanyaan spesifik tentang denda/SOP secara akurat.
3. **Multi-tenant filtering di level ChromaDB** — Saat ini filtering rental_id dan kota dilakukan di MySQL. Untuk RAG sejati, metadata filtering juga harus dilakukan saat vector search di ChromaDB agar tenant isolation terjaga di level retrieval.

Catatan Keterbatasan Analisis

- Tidak bisa mengakses kode Python Flask service di HuggingFace Spaces (<https://iZikra-iZikraSpaces.hf.space>) — verifikasinya hanya berdasarkan response pattern dan env variable
- Tidak bisa menjalankan kode untuk test
- Tidak bisa melihat isi vendor/ directory secara detail
- ChromaDB server (jika ada) tidak bisa diakses langsung dari sini

Analisa dilakukan pada: 2026-05-18

Sumber data: FTP access ke <ftpupload.net/htdocs/>